

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Operators

Primitive types in Haskell

Basic operations on lists

Tuples and Records

Polymorphic types and Overloading

Identifiers and constants in Haskell

Definition

A letter followed by any number of letters, digits, `_`, `'`

Example

```
supercalifragilisticexpialidocious  
america'  
catch_22  
Nothing
```

Example (Haskell, batch style)

```
pi = 3.14
```

Observation

No function can change the value assigned to it.

Operators

Definition

An infix operator is a function written **between** its arguments. This is done for syntactic sugaring.

Example

```
(|!&) :: Bool -> Bool -> Bool  
infixl 6 |!&  
x |!& y = (x || y) && not (x && y)
```

```
*Main> True |!& True  
False
```

Operator precedence

Example (number just after `infix`)

```
infixl 6 |+|
```

```
x |+| y = "(" ++ x ++ "+" ++ y ++ ")"
```

```
infixl 5 |*|
```

```
x |*| y = "(" ++ x ++ "*" ++ y ++ ")"
```

```
*Main> "2" |+| "3" |*| "4"
```

```
"((2+3)*4)"
```

Operators

Example (`infixl`: left associativity)

```
infixl 6 ##
```

```
x ## y = "(" ++ x ++ "+" ++ y ++ ")"
```

```
Prelude> "1" ## "2" ## "3"
```

```
"((1+2)+3)"
```

Example (`infixr`: right associativity)

```
infixr 6 ##
```

```
x ## y = "(" ++ x ++ "+" ++ y ++ ")"
```

```
Prelude> "1" ## "2" ## "3"
```

```
"(1+(2+3))"
```

Operators

Note

An infix operator could be used as prefix function.

Example (Haskell addition)

```
1 + 2
```

same as

```
(+) 1 2
```

Example (Haskell custom operator)

```
"1" ## "2"
```

```
(##) "1" "2"
```

Example (Haskell division)

```
3 `div` 2
```

```
div 3 2
```

Types and type checking

Definition (Type)

Type = collection of related values + functions over them

Remark

Strongly typed languages require each entity to have a type assigned and enforce strict typing rules for operations.

Definition (Type inference)

Type inference = find out the type of an expression

Definition (Static v. Dynamic type checking)

- ▶ Static type checking = verify type safety at compile time
- ▶ Dynamic type checking = verify type safety at runtime

Primitive types: Integers

Integers in Haskell

- ▶ `Int` / `Integer`: fixed / arbitrary precision integers
- ▶ binary operations: `+` `-` `*` `'div'` `'mod'`
- ▶ relational operators: `<` `>` `<=` `>=` `==` `/=`
- ▶ `Integral` is sort of "supertype" for both `Int` and `Integer`

Example (Haskell Int operators)

```
Prelude> 1 + 2
```

```
3
```

```
Prelude> 1 < 2
```

```
True
```

```
Prelude> 3 'div' 2
```

```
1
```

Primitive types: Real

Real type in Haskell

- ▶ `Float / Double`: simple, respective double precision
- ▶ **operations**: `+` `-` `*` `/`
- ▶ **relational operators**: `<` `>` `<=` `>=` `==` `/=`
- ▶ `1.4142135` (`Float`)
`1.41421356237` (`Double`) `-10.0` `1.0E-2`
- ▶ `Fractional` is sort of "supertype" for both `Float` and `Double`
- ▶ `Num` is sort of "supertype" for both `Integral` and `Fractional`

Primitive types: Boolean

Boolean type in Haskell

- ▶ **type:** `Bool`; **values:** `True`, `False`
- ▶ **logical operations:** `not`, `&&`, `||`
- ▶ **conditional expression:** `if E then ET else EF`

Character and string

Example (Char: between quotes ('))

```
'a', 'b', ..., 'z', '3'  
'\n', '\\'
```

Example (String: between double quotes ("))

```
"abc", "a1b2", ""
```

String Operators

1. relational: <, >, <=, >=, ==, /=
2. concatenation: ++

A little bit of lists and primitive functions on lists

Definition (List)

A list is a sequence of values of a specific type.

Example (Haskell)

```
Prelude> :set +t --ask ghci to tell the type
Prelude> [1,2,3] :: [Integer]
[1,2,3] :: [Integer]
```

Observations

1. the **length** of the list is not limited
2. the **types** of list elements are not restricted
3. however, the elements must have the **same** type

A little bit of lists and primitive functions on lists

Example (Haskell)

1. `Prelude> head [1,2,3]`
`1`
2. `Prelude> tail [1,2,3]`
`[2,3]`
3. `Prelude> null [1,2,3]`
`False`
4. `Prelude> 1:[2,3]`
`[1,2,3]`
5. `Prelude> [1,2] ++ [3,4]`
`[1,2,3,4]`
6. `Prelude> length [11,12,13]`
`3`
7. `Prelude> [11,12,13] !! 1`
`12`
8. `Prelude> elem 12 [11,12,13]`
`True`

Tuples

Definition (Tuple)

A finite sequence of components of possibly different types.

E.g.: `(1, '2', "four")`

Remarks

- ▶ tuples: enclosed in parentheses
- ▶ components: comma-separated
- ▶ component order matters (semantics connected to component position)
- ▶ use case: a function which needs to return several values (e.g., a complex number)

Building and accessing tuples in Haskell

Example (A person has name, age, gender, address)

```
--a tuple which stores these components
type Person = (String, Int, Char, String)
--Person is just an alias for the tuple

--let me introduce Brad: tuple constructor
a1 :: Person
a1 = ("Brad", 60, 'M', "L. A.")

Main> :t a1  --the system "knows" the new type
a1 :: Person

--deconstruct a1 to find out info on him
(thename, theage, thegender, theaddress) = a1
--constructors do the deconstruct job as well
```

Building and accessing tuples in Haskell

Example (A person has name, age, gender, address)

```
--deconstruct a1 to find out info on him  
(thename, theage, thegender, theaddress) = a1
```

```
Main> theage  
60
```

Union/sum/enumerate type

Example (Introduce a brand new type: Gender (male/female))

```
data Gender =          -- the new type's name
  M | F                -- alternative constructors
  deriving (Show)      -- values can be displayed

--a tuple stores personal info pieces
type Person = (String, Int, Gender, String)
```

Building and accessing tuples in Haskell

Example (Add and extract personal info)

```
--let me introduce Brad
a1 :: Person
a1 = ("Brad", 60, M, "L.A.")

--let's learn something about a1
(thename, theage, thegender, thelocation) = a1
```

Printing components of a tuple

Example (The system can print a String...)

```
Main> thelocation  
"L.A."
```

Example (The system struggles to print a Gender...)

```
Main> thegender  
M
```

Printing components of a tuple

Example (...so we should teach him how to improve)

```
printGender :: Gender -> String
-- the function type may use the new type

--define function by cases
printGender s = case s of
    M -> "Male"
    F -> "Female"

Main> printGender thegender
"Male"
```

Building and accessing records in Haskell

Example (A person has name, age, sex)

```
--a record stores info in a set of NAMED fields
--semantics connected to field names
data Prs = Prs
    { name  :: String
    , age   :: Int
    , sex   :: Char
    }

--let me introduce Brad: record constructor
a1 = Prs { name = "Brad", age = 60, sex = 'M' }
```

Building and accessing records in Haskell

Example (A person has name, age, sex)

```
--let me introduce Brad: record constructor  
a1 = Prs { name = "Brad", age = 60, sex = 'M' }
```

```
Main> :t age --field names work as getters
```

```
age :: Prs -> Int
```

```
Main> age a1
```

```
60
```

Polymorphism

Strong typing versus Weak typing

Strong typing provides safety; weak typing provides flexibility.
Polymorphism aims to reconcile these.

Definition

Polymorphic entity = can have more than one type

Definition

Polymorphic function = a function based on **type variables**
(which can be further instantiated to create specific types).

Polymorphism

Example

```
head :: [a] -> a
```

Example (Type variable `a` gets instantiated to `Char`)

```
Prelude> head ['a', 'b', 'c']  
'a'
```

Example (Type variable `a` gets instantiated to `[Char]`)

```
Prelude> head ["CFR", "FCSB", "U"]  
"CFR"
```

Remark

One behavior for the function, different argument types

Type inference

Definition

An expression is **well typed** if the type checker is able to assign values to each of its type variables in a non-contradictory way.

Inference rule for type inference

$$\frac{f :: A \rightarrow B \quad e :: A}{(f\ e) :: B}$$

Explanation

Let's assume function f maps entities of type A into entities of type B and e is an entity of type A . If f is passed e as an argument, the returned entity will have type B .

Type inference example

Example

```
facti (n, p) = if (0 :: Int) == n then p
              else facti (n-1, n*p)
```

Type inference

`0 == n` is `Int` comparison (because both the following hold:

`(0 :: Int) ==` :: `Int -> Bool` and `0 :: Int`)

→ `n` is of type `Int`

AND `n * p` is `Int` multiplication

→ `p` is of type `Int`

AND `p` is of type `Int`

→ `facti` is of type `(Int, Int) -> Int`

Overloaded functions

Definition

Overloaded function = a polymorphic function whose type contains one or more constraints.

Example (constraint: t must be of type number (Num))

```
mysum [] = 0  
mysum (x:xs) = x + mysum xs
```

```
Prelude> :type mysum  
mysum :: Num t => [t] -> t
```

Remark

Same name for the function, different implementations.

Some type classes in Haskell and their supported methods

- ▶ `Eq` (equality types): their values can be compared using `==` and `/=` (e.g. `Int`, `Char`, lists with elements in `Eq`)
- ▶ `Ord` (ordered types): those in `Eq` which can be ordered using `<`, `<=`, `>`, `>=`, `min`, `max`
- ▶ `Show` (showable types): those whose values can be converted to strings (and displayed) by using `show`
- ▶ `Num` (numeric types): numbers, on which we can apply `+`, `-`, `*`, `negate`, `abs`, `signum`
- ▶ `Integral`: types which are instances of `Num`, on which we can apply `div`, `mod` (e.g. `Int`, `Integer`)
- ▶ `Fractional`: types which are instances of `Num`, on which we can apply `/`, `recip` (e.g. `Float`, `Double`)

Polymorphic equality

Definition

An **equality type** is a type endowed with an equality predicate. (which is not available for *every* object in FP, as we focus on semantics, not equality of memory locations).

Example (Haskell: `Eq a`)

```
Prelude> :type (==)
(==)  :: Eq a => a -> a -> Bool
```

Polymorphic equality

Example (Seek an element in a list)

```
myelem _ [] = False
myelem e (x:xs) = if (e==x) then True
                  else myelem e xs
```

```
Prelude> :t myelem
myelem :: Eq t => t -> [t] -> Bool
```

Equality types

This function could be called **only** on lists whose elements are of equality types, e.g. `Int`, `String`, `Char` etc.

Some more type inference examples

Example (a pair of arguments, parentheses)

```
type Vector = (Int, Int)
gcd4 :: Vector -> Int
gcd4 (m,n) = if m == 0 then n
            else gcd4 (n `mod` m, m)

gcd4 :: Vector -> Int
```

Example (a pair of arguments, parentheses)

```
type Vector = (Int, Int)
gcd3 (m,n) = if m == 0 then n
            else gcd3 (n `mod` m, m)

gcd3 :: Integral t => (t, t) -> t
```

Some more type inference examples

Example (a pair of arguments, parentheses)

```
gcd2 (m,n) = if m == 0 then n
           else gcd2 (n `mod` m, m)
```

```
gcd2 :: Integral t => (t, t) -> t
```

Example (2 arguments, no parentheses)

```
gcd1 m n = if m == 0 then n
           else gcd1 (n `mod` m) m
```

```
gcd1 :: Integral t => t -> t -> t
```

Introducing new types

Example (`type`: **alias** for an existing type)

```
type Astring = [Char]
```

Example (`data`: **build a brand new type via enumeration**)

```
data Color = Red | Green | Blue
```

new type type constructors ← names in CAPITALS!

Example (`Maybe`: **constructors might have arguments**)

```
data Maybe a = Nothing | Just a
```

A value of this type either stores a value of type `a` (represented as `Just a`), or is just empty (represented as `Nothing`).

Introducing new types

Definition (Sum/union/enumerate types)

Are defined using alternatives.

Example (Sum type)

```
data Gender = M | F
data Maybe a = Nothing | Just a
```

Definition (Product types)

Are defined in a "cartesian product"-style.

Example (Product type)

```
type Person = (String, Int, Gender, String)
data Pers = Pers { name :: String, age  :: Int }
```

A 2×2 matrix

Problem

Write an operator $\star$ for 2×2 matrix multiplication

Introducing a new data type: a 2×2 matrix

```
data Matrix2x2 = Mat2 ((Integer, Integer),  
                        (Integer, Integer))  
    deriving (Show)
```

A 2×2 matrix

The new type is `Num` and implements `*`

```
instance Num Matrix2x2 where
  Mat2 (ali1, ali2) * Mat2 (bli1, bli2) =
    let
      (a11, a12) = ali1 -- deconstruct lines
      (a21, a22) = ali2
      (b11, b12) = bli1
      (b21, b22) = bli2
    in
      Mat2 ((a11*b11+a12*b21, a11*b12+a12*b22),
            (a21*b11+a22*b21, a21*b12+a22*b22))
```

A 2×2 matrix

A new operator: matrix at number

```
infixr 8 |^|
```

```
Mat2 (ali1, ali2) |^| n =  
  if (n==0) then Mat2 ((1,0), (0,1))  
  else Mat2 (ali1, ali2) *  
        Mat2 (ali1, ali2) |^| (n-1)
```

Summary

- ▶ Infix operators provide syntax sugaring
- ▶ `head`, `tail`, `null`, `:`, `++`, `!!`, `elem` provide a basic repertoire of list manipulation functions
- ▶ Tuples and records increase flexibility and readability
- ▶ In Haskell we have the usual basic types (Integer, Bool, Char, String) and a sophisticated type classes system
- ▶ New types can be introduced as sum or product types, by deriving etc.